



可验证随机函数的动态PBFT共识算法

吴帅珂, 黄 慧*

(闽南师范大学计算机学院, 福建 漳州 363000)

摘 要: 共识算法是区块链系统的重要组成部分,其有效性直接影响到整个系统的实用性。实用拜占庭容错共识算法(practical byzantine fault tolerance, PBFT)是联盟链中被广泛应用的一种共识算法。目前的PBFT存在着视图切换开销过大、节点无法动态进出共识系统以及系统健壮性较差等问题。为此,提出一种基于可验证随机函数(verifiable random function, VRF)的动态拜占庭容错共识算法(adaptive dynamic byzantine fault tolerance, AD-BFT)。在该算法中,使用VRF算法优化原有视图切换过程,减少视图切换过程的通信量,提高共识吞吐量。设计了一套节点动态更新机制,用于处理节点动态进出共识网络的相关事务。最后,采用熔断机制,可以在系统出现节点大量脱机时防止共识系统崩溃,增强系统健壮性。实验结果与分析表明,所提出的AD-BFT相较于实用拜占庭容错共识算法(practical byzantine fault tolerance, PBFT)具有更低的共识时延与吞吐量,且能够在共识过程并行处理成员变更。

关键词: 共识算法;区块链;联盟链;PBFT;VRF

中图分类号: TP391 **文献标志码:** A **文章编号:** 2095-7122(2025)01-0025-16

Dynamic PBFT consensus algorithm based on verifiable random functions

WU Shuaike, HUANG Hui*

(School of Computer Science and Engineering,
Minnan Normal University, Zhangzhou, Fujian 363000, China)

Abstract: The consensus algorithm is an important part of the blockchain system, and its effectiveness directly affects the practicality of the whole system. Practical Byzantine Fault Tolerance(PBFT) is a widely used consensus algorithm in consortium blockchains. The current PBFT has problems such as excessive view-change overhead, nodes not being able to dynamically move in and out of the consensus system, and poor system robustness. This work proposes an adaptive dynamic Byzantine fault Tolerance (AD-BFT) algorithm based on verifiable random Function(VRF). In this work, the VRF algorithm is used to optimize the original view-change process, reduce the communication volume of the view-change process, and improve the consensus throughput. In this work, a set of node dynamic update mechanisms is designed to deal with the transactions related to the dynamic entry and exit of nodes into and out of the consensus network. Finally, a circuit breaker mechanism is used to prevent the consensus system from collapsing when a large number of nodes are offline in the system and enhance the robustness of the system. Experimental results and analysis show that the pro-

收稿日期: 2024-10-12

基金项目: 福建省自然科学基金项目(2023J01920)

作者简介: 吴帅珂(1999—),男,河南平顶山人,硕士生。

*通信作者. E-mail: hhui323@163.com

posed AD-BFT has lower consensus latency and throughput compared with practical Byzantine fault tolerance (PBFT) and can process membership changes in parallel during the consensus process.

Key words: consensus algorithm; blockchain; consortium blockchain; PBFT; VRF

区块链技术最初是在 2008 年由一位化名为中本聪的研究者提出的^[1],是支撑比特币等加密货币运行的核心技术。本质上,区块链是一个结合了密码学算法的分布式数据库,通过多个节点之间的共识机制来记录和验证交易数据^[2]。在多个领域均有广泛应用^[3],根据应用场景不同可将区块链分为公有链、私有链和联盟链^[4]。其中联盟链允许成员之间有不同的数据访问权限,这种结构既保证了部分去中心化的特性,又提供了必要的集中控制,因此联盟链被视为当前区块链应用的主流趋势。共识机制是区块链技术的必要组成部分,其核心功能是定义了网络中的各个节点如何对交易的有效性达成一致,以此来维护区块链的完整性和安全性^[5]。实用拜占庭容错共识算法(practical byzantine fault tolerance, PBFT)是经典的拜占庭容错共识算法,成功的将通信复杂度从指数级降低至多项式级,并因其简单、高效、稳定和可扩展的特点在多个领域得到了广泛应用。

然而,尽管 PBFT 算法已取得了显著进展,却仍存在一些问题需进一步解决,因此许多研究者对 PBFT 共识算法进行了各种改进。这些优化方案大致可分为两类:保持副本节点个数不变的同时降低参与共识的节点数量和优化共识流程中的结构。InDPa 等^[6]将节点分为信任节点和故障节点,故障节点无法投票,且大部分投票会选择声誉较高的节点,所以可有效减少参与投票的共识节点数量,通过降低通信开销来降低共识的整体通信成本。Ding 等^[7]通过将共识节点分组,在组内使用聚类算法降低组内通信产生的能量消耗,等待组内共识完成后,再在组间进行共识,并在验证过程采用了无证书的聚合签名来减少通信和计算的开销。Liu 等^[8]借助流水线技术,随机选择部分节点来参与共识投票,由于减少了实际参与共识的节点数量,因此可以显著提高共识系统的一致性效率与吞吐量。Tang 等^[9]提出了无权限、可扩展的 Hedera 共识算法,采用茎叶链的结构,茎链采用 PoC 共识算法,节点获得出块权的概率与其投入的存储空间在网络总存储空间中所占的比例成正比,并负责选举下一轮叶链的委员会成员,委员会成员在叶链上进行交易排序、共识和打包, Hedera 具有无权限、安全等优点。Guo 等^[10]使用了多主节点机制来解决单点故障,引入了一个数据同步模型以确保视图更改后块数据的一致性,并通过引入动态声誉模型来提升共识鲁棒性。Hao 等^[11]通过在每个视图中引入两个领导者,来减少视图切换过程所需要经历的延迟,进而提升共识效率。Xiao 等^[12]设计了一种考虑节点完成率、一致衰减和节点行为的信用评估模型,动态地选择非拜占庭节点进行共识,来提高系统的可靠性和安全性。Fathi 等^[13]根据节点的网络位置与性能,将共识节点分片,片内使用 IBFT 来进行共识,提高了物联网共识的可伸缩性与吞吐量。

这些改进对于提高共识吞吐量以及共识开销方面取得了显著进展,但在面对高频交易场景、物联网、车联网等特定应用场景时还有一些尚未解决的问题,例如共识系统无法感知节点数量,节点无法动态进出系统,视图切换过程延迟较高,大规模节点宕机时无法保证可用性等。为了解决这些问题,融合可验证随机函数改进领导者的选举规则,使得领导者的选举完全随机且可验证,可以避免预谋攻击。优化视图切换协议过程,以减少通信成本并提高数据吞吐量。设计了节点的动态进出机制,以改善算法的可扩展性。并采用熔断机制,以确保系统遇到大规模

节点宕机的极端情况时仍能保证共识可用。

1 相关知识

1.1 系统架构

如图1所示,区块链平台整体上可划分为网络层、共识层、数据层、智能合约层和应用层5个层次^[4]。网络层是区块链的最底层,主要负责区块链节点之间的通信和数据传输,例如节点发现和数据同步。共识层主要负责执行区块链网络中的共识机制,确保数据的一致性。数据层主要负责数据的存储和加密。智能合约层允许在区块链上运行可自动执行的程序,对链上数据进行处理,满足业务需求。应用层是直接面向最终用户的层面,为用户提供具体的应用服务。

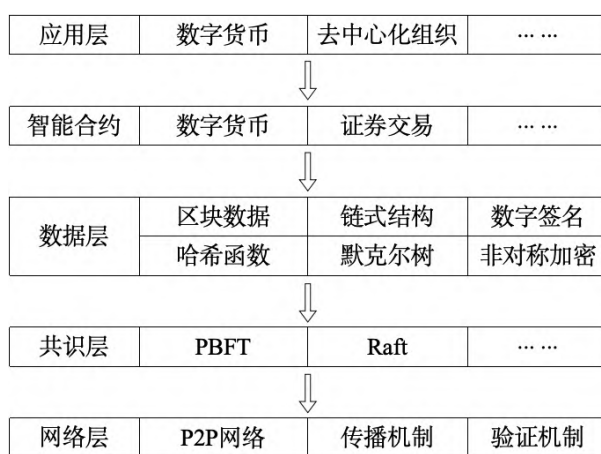


图1 系统架构

Fig. 1 System architecture

1.2 PBFT 共识算法

PBFT 共识算法运行在共识层,它将拜占庭容错算法从指数级的通信复杂度降低到多项式级,为拜占庭容错算法真正应用于工程领域提供了可能^[14]。在 PBFT 中节点分为主节点和副本节点,其中主节点负责交易验证、打包区块等,副本节点参与共识过程,与主节点一起确保共识一致性。在正式执行共识之前,PBFT 算法需要选择一个副本节点作为主节点,主节点 P 由式(1)计算得出,其中 V 为视图编号, N 为共识节点总数。

$$P = V \bmod |N|。 \quad (1)$$

即 PBFT 共识算法最多可在 $f \leq (N-1)/3$ 时实现容错,其中 f 是拜占庭节点的数量。PBFT 的共识过程包括 request、pre-prepare、prepare、commit 和 reply 共 5 个阶段。如图 2 所示,client 为客户端,node 0 为当前视图主节点,副本节点编号分别为 1、2 和 3,其中节点 3 处于宕机状态模拟节点故障。在 request 阶段,客户端 client 将请求发送给主节点,主节点对消息进行验证,验证通过后在 pre-prepare 阶段向所有副本节点广播该请求。副本节点收到消息并对其进行验证,验证通过后,在 prepare 阶段将带有节点自身签名的消息再次广播给所有副本节点。当所有副本节点收集到超过 $2f+1$ 个不同节点对这笔交易的签名信息后,在 commit 阶段向所有节点发送确认信息,当节点收集到超过 $2f+1$ 个不同节点的确认信息后,执行这笔交易并将交易执行结果发送给客户端 client,当客户端收集到来自 $f+1$ 个不同节点所回复的相同结果时,则认为请求被成功执行。

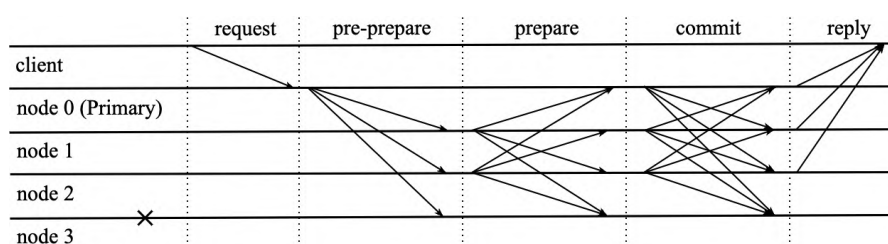


图2 PBFT 的共识过程

Fig. 2 Consensus process of PBFT

1.3 VRF

可验证随机函数 VRF 是一种伪随机数生成函数,具有可验证性、唯一性、随机性的特点^[15]。在 AD-BFT 中,将节点 node i 的私钥 sk_i 和种子 $seed$ 输入 VRF_VAL 函数,即可得到一个伪随机数 $vrf_randnum$ 和证明 vrf_proof 即

$$vrf_randnum, vrf_proof = VRF_VAL(sk_i, seed)。(2)$$

所有节点都可以将节点 node i 的公钥 pk_i 、 $seed$ 和计算得出的 $vrf_randnum$ 、 vrf_proof 输入 VRF_VER 函数,根据输出结果为 True 或 False 来验证伪随机数生成节点的身份即

$$VRF_VER(pk_i, seed, vrf_randnum, vrf_proof)。(3)$$

1.4 XOR 距离

XOR 指导或算法,AD-BFT 算法通过 XOR 来计算一个逻辑上的距离 L ,而非地理上的距离,使用 SHA256 来计算一段数据的哈希值即

$$L(x, y) = SHA256(x) XOR SHA256(y)。(4)$$

每个节点都有一个随机生成的长度为 256 bit 的无符号二进制数来作为节点 ID。假设节点 A 的 ID 为 $A.id$,节点 B 的 ID 为 $B.id$,那么节点 A 与节点 B 的距离 $L(A, B) = SHA256(A.id) XOR SHA256(B.id)$ 。因此每一个节点都可以计算出两节点间的距离或自身节点 ID 与最新稳定检查点的距离(稳定检查点在此处可理解为一段二进制数据,在后面的节 2.2 中介绍),并以此作为节点竞选成为新的领导者的依据。XOR 距离具有以下特征^[16]:

- 1) 节点 ID 随机生成且使用 SHA256 计算哈希值,所以任意两节点间的距离是随机的;
- 2) 若 $A \neq B$,则 $L(A, B) > 0$;
- 3) 节点到自身的距离为 0,即 $L(A, A) = 0$;
- 4) 节点 A 到节点 B 的距离,与节点 B 到节点 A 的距离相等,即 $L(A, B) = L(B, A)$;
- 5) 因为 $L(A, B) XOR L(B, C) = L(A, C)$,且对于所有的 $a \geq 0, b \geq 0$,都有 $a + b \geq a XOR b$,所以可得 $L(A, B) + L(B, C) \geq L(A, C)$ 。

由于共识系统中节点数量在一个视图内不变,且最新稳定检查点是确定的,因此可保证在使用 XOR 计算节点 ID 与最新稳定检查点之间的距离时,能够找到距离最近的节点,并可由所有节点快速验证。

2 AD-BFT 共识算法

AD-BFT 共识算法运行在共识层,以 PBFT 为基础进行改进。

1) 提高共识效率。在保证活性以及公平性的前提下,通过优化领导者选举规则以及视图切换协议,降低视图切换过程对共识过程的影响,从而提高共识效率。

2) 动态进出。在不影响共识系统安全性的前提下,保证节点可以主动进出共识系统。

3) 提高可用性。当共识系统出现大规模副本节点宕机时,仍能保证共识的可用性。

AD-BFT主要由6个核心协议组成:一致性协议、检查点协议、领导者选举协议、视图切换协议、动态进出协议和熔断机制。一致性协议用来保证共识日志的一致性,是共识算法的核心。检查点协议可以有效降低节点的存储开销。领导者选举协议保证了随机且公平的选举领导者。视图切换协议用来保证共识系统的活性,当出现领导者作恶时,通过视图切换协议更换领导者,进入新的视图。动态进出协议为共识节点提供了动态进出的能力。熔断机制提高了共识健壮性,帮助系统在遇到大规模节点宕机时仍能保证系统可用。

2.1 一致性协议

一致性协议是共识算法的核心,联盟链通过定期将交易信息打包成区块来维护账本信息,共识算法通过一致性协议来保证交易信息的一致性和完整性。

AD-BFT的一致性协议同PBFT基本一致,包括 request、pre-prepare、prepare、commit 和 reply 共5个阶段。如图3所示,client为客户端,node 0~node 2分别为领导者DP、BDP、BBDP,node 3为副本节点。客户端在 request 阶段向主节点 node 0 发送请求,主节点在验证消息完成后,在 pre-prepare 阶段向所有副本节点广播该请求。副本节点收到消息验证通过后,在 prepare 阶段将带有节点自身签名信息的信息进行广播。当所有副本节点在收到超过 $2f+1$ 个不同节点对这笔交易的签名信息后,在 commit 阶段向所有节点发送确认消息,当节点收集到超过 $2f+1$ 个不同节点的确认信息后,执行这笔交易并将交易执行结果发送给客户端 client,当客户端收集到来自 $f+1$ 个不同节点所回复的相同结果时,则认为请求被成功执行。

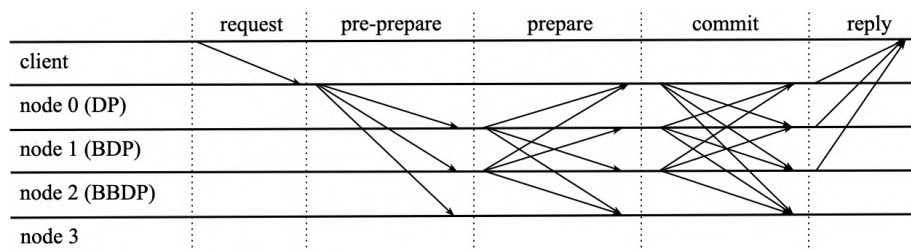


图3 AD-BFT的共识过程

Fig. 3 Consensus process of AD-BFT

2.2 检查点协议

共识算法运行过程中,副本节点会产生大量的日志,导致节点存储开销过大。检查点协议可以安全的去除不必要的日志记录,降低存储开销。

当一致性协议执行完成生成区块后,会向所有副本节点发送 CHECK-POINT 消息,其中包含有最后一个请求的序号 n ,上一个稳定检查点的区块号以及状态摘要(上一个稳定检查点到当前生成区块之间所有区块的哈希值)。副本节点在验证 CHECK-POINT 消息成功后,会将此消息进行签名并在共识系统中广播。若副本节点收到来自不同节点的超过 $2f+1$ 个正确的检查点信息,则这 $2f+1$ 条信息就是检查点的正确性证明,具有证明的检查点称为稳定检查点,而最后生成的稳定检查点,就称为最新稳定检查点。此时便可以去除日志中所有请求序号小于 n 的日志信息。稳定检查点代表了系统在某个时刻达成的一致性状态,并且这个状态已经被足够多的节点确认和记录。

2.3 领导者选举

如图4所示,AD-BFT算法在每一个视图中的领导者共有3个,分别为DP(designated primary)、BDP(backup designated primary)和BBDP(backup for backup designated primary),其中:BDP是DP的备份,BBDP是BDP的备份。



图4 领导者分类

Fig. 4 Primary classification

当共识系统中还没有领导者时,每个副本节点都会进入选举过程,尝试竞选成为DP。DP选举过程如算法1所示,其中latest_stable_checkpoint指最新稳定检查点,self指节点自身,all_node指所有节点集合,support_list表示收到的投票列表。每个副本节点均执行算法1,首先使用公式(4)计算节点自身ID与最新稳定检查点的距离 L ,以此作为节点竞选DP的凭证,在共识系统中广播拉取选票,若收到其他距离更小的节点拉取选票,则会将已经收到的选票与自身的选票一起发送给更优的节点,每个节点只能投票一次,已经投票的节点不会再成为领导者。当所有的节点收敛完成时,距离最小的节点会成为新一任的DP,若出现距离相等的情况,则ID更小的节点更优,随后新一任的DP节点增加视图号,广播NEW-VIEW消息完成本轮选举,并时刻向所有副本节点发送KEEP-ALIVE消息。

算法1 DP选举算法

输入:latest_stable_checkpoint, self, all_node, support_list
输出:primary_node, timestamp

```

1: primary_node ← self;
2: for each node node[i] in all_node do
3:   L ← ( SHA256(node[i].node_id) XOR SHA256( latest_stable_checkpoint ) );
4:   if ( SHA256( primary_node.id ) XOR SHA256( latest_stable_checkpoint ) ) < L then
5:     primary_node ← self;
6:     support_list.add( node[i] );
7:     return primary_node, timestamp;
8:   elif ( SHA256( primary_node.id ) XOR SHA256( latest_stable_checkpoint ) ) = L then
9:     if self.node_id < node[i].node_id then
10:      primary_node ← self;
11:      support_list.add( node[i] );
12:      return primary_node, timestamp;
13:     else
14:      primary_node ← node[i];
15:      return primary_node, timestamp, support_list;
16:     end
17:   else then
18:     primary_node ← node[i];
19:     return primary_node, timestamp, support_list;
20:   end
21: end

```

算法2 BDP选举算法

```

输入: self, seed, all_node
输出: BDP, vrf_randnum, vrf_proof, timestamp
1: vrf_randnum, vrf_proof  $\leftarrow$  VRF_VAL( self.sk, seed );
2: BDP  $\leftarrow$  node[0];
3: for each node node[i] in all_node do
4:   L  $\leftarrow$  ( SHA256( node[i].node_id ) XOR SHA256( vrf_randnum ) );
5:   if ( SHA256( BDP.id ) XOR SHA256( vrf_randnum ) ) > L then
6:     BDP  $\leftarrow$  node[i];
7:   elif ( SHA256( BDP.id ) XOR SHA256( vrf_randnum ) ) = L then
8:     if BDP.node_id > node[i].node_id then
9:       BDP  $\leftarrow$  node[i];
10:    end
11:  end
12: end
13: return BDP, vrf_randnum, vrf_proof, timestamp;

```

BDP的选举过程如算法2所示,其中self指DP节点自身,seed指VRF中的种子,集群中每个节点都有一个相同的group-id作为种子seed,其中group-id为共识系统中所有节点ID集合的哈希值,用于熔断机制中区分不同的集群。DP首先使用公式(2)计算得出随机数vrf_randnum与证明vrf_proof。随后DP选择距离随机数vrf_randnum最近、节点ID最小且非自身的节点成为所选择的BDP。当DP选择出BDP后,BDP保留DP所发送的vrf_randnum与vrf_proof,并时刻向DP发送KEEP-ALIVE消息,随后BDP使用同样的方法选择出BBDP。

在视图切换过程中,BDP和BBDP的节点ID不会向所有副本节点公开,当前视图中的BDP也只会需要在视图切换时公开vrf_randnum和vrf_proof来证明自己的身份。其余情况下,BDP与BBDP均充当普通副本节点,所有副本节点也只接受DP所发送的提案。

2.4 视图切换

AD-BFT协议在出现领导者节点宕机或做恶时,通过切换视图选择新的领导者保证活性。如图5所示,假设此时为视图 v ,若领导者DP在 t_1 时刻出现宕机,经过一段时间后,BDP在 t_2 时刻监测到DP宕机,则向所有副本节点广播NEW-VIEW消息,并切换成为 $v+1$ 视图的DP,消息包含有 v 视图中DP所发送的vrf_randnum和vrf_proof,其他副本节点可据此判断出是否有节点冒充BDP,由于无法伪造VRF生成的随机数,因此可避免DP指定节点成为BDP。副本节点收到NEW-VIEW消息验证通过后,进入下一视图,便完成视图切换过程(图中 t_3 时刻)。随后BBDP成为新的BDP,并可在正常共识过程中选出 $v+1$ 视图的BBDP。若DP主动作恶,那么BDP在 t_2 时刻发现后,会在共识系统内广播NEW-VIEW消息,消息中包含有DP主动作恶所发出的信息,并切换成为 $v+1$ 视图的DP。相较于原有的PBFT,可以节省下一任领导者收集普通副本发送视图切换消息的过程,更快的完成视图切换,并且领导者的选择完全随机,可以避免预谋攻击。

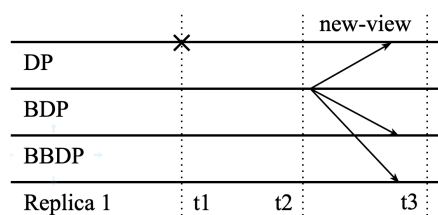


图5 视图切换 1

Fig. 5 View change 1

如图6所示,假设主节点DP在 t_1 时刻发生宕机,在正常情况下,会在 t_3 时刻完成视图切换,但此时若BDP也宕机,则在 t_3 时刻其他的副本节点会因没有收到BDP的NEW-VIEW消息而广播VOTE消息,当BBDP在 t_4 时刻收集到了超过 $2f+1$ 个不同节点的VIEW-CHANGE消息后,由BBDP发送NEW-VIEW消息进行视图切换,成为新视图的DP。若BDP作恶,则其他副本节点在 t_3 时刻发现BDP作恶后,在共识系统内广播VOTE消息,其中包含有BDP主动作恶所发出的信息,当BBDP收集到超过 $2f+1$ 个不同节点的VIEW-CHANGE消息后,由BBDP发送NEW-VIEW消息进行视图切换,成为新视图的DP。为了避免占用不必要的通信资源,这个过程只有在DP与BDP同时出现宕机或作恶时才会触发,所以不会影响到正常的视图切换过程。

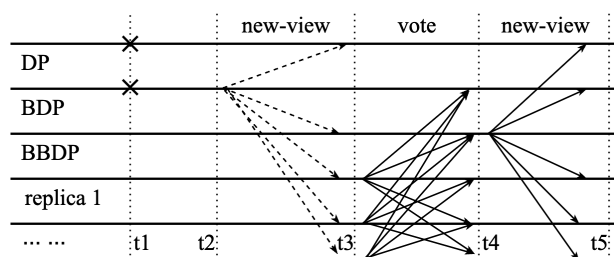


图6 视图切换 2

Fig. 6 View change 2

若共识网络中DP、BDP、BBDP均宕机或主动作恶,由于这种概率极小,因此当副本节点确认领导者节点均宕机后,触发熔断机制,此部分在2.6中描述。

2.5 动态进出

与任何静态分布式系统一样,静态BFT系统存在着实际限制^[17]。当网络中的某个节点崩溃或被认为是有故障(包括主动作恶)时,系统的安全性下降,而恢复一个节点往往需要时间,更重要的是恢复有缺陷的节点并不总是可行的(例如系统中残存有病毒或出现硬件故障),此时使用一个全新的节点来替代有缺陷的节点是一个更好的办法^[18]。而传统的BFT共识算法中的成员资格都是静态的,副本节点无法动态的加入或离开系统。因此AD-BFT要求在不影响共识情况下,允许节点主动加入或退出共识系统。

1) 新节点申请加入共识系统。

图7为外部新节点加入现有共识系统的过程。node-new为请求加入的新节点,node 0为当前视图的DP,node 1~node 3为副本节点,具体步骤如下。

request: node-new节点向DP发送申请加入共识系统的REQUEST消息,包含节点自身ID、公钥信息以及时间戳。

sync: DP收到REQUEST消息后,向node-new节点发送SYNC消息,包含当前集群的group-

id、现有节点数量、最新稳定检查点、视图号以及区块信息。

ready-join: node-new 节点收到 SYNC 消息后保存对应信息并同步本地账本。随后重新向 DP 发送 REQUEST 消息,包含自身信息和获取到的 group-id、当前区块信息、最新稳定检查点哈希值等摘要信息。

broadcast: DP 收到消息在验证通过后对此申请进行签名并在共识系统中广播,等待投票信息。

commit: 副本节点收到消息,在验证通过后将投票信息进行签名并回复至 DP。

reply: 当 DP 收到来自 $2f+1$ 个不同节点的同意投票信息后,更新当前共识系统最新节点数量,并将确认信息在系统中广播,包括当前共识系统的最新节点数量以及所有节点的公钥信息, node-new 收到此消息后完成加入过程,以副本节点身份参与后续共识流程。

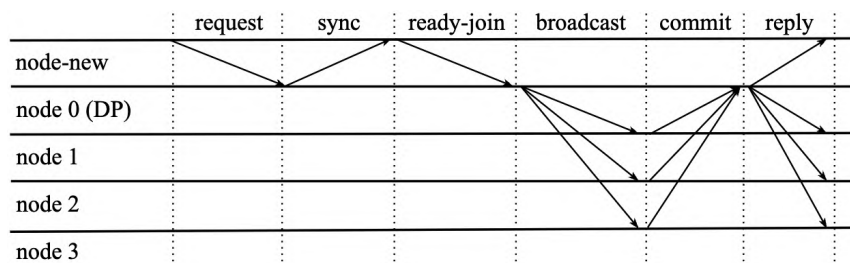


图7 新节点加入过程

Fig. 7 Joining process of new nodes

2) 已有节点主动申请退出共识系统。

图8为内部已有节点主动退出的过程, node-quit 为请求退出的节点,具体流程如下。

request: node-quit 节点向 DP 发送申请退出共识系统的 REQUEST 消息,包含有节点自身的签名。

broadcast: DP 收到 REQUEST 消息,确认节点身份并验证通过后,对此消息进行签名并在共识系统中广播,等待意见信息。

commit: 副本节点收到消息并在验证通过后,将自己的意见信息进行签名发送至 DP。

reply: 当 DP 收到来自 $2f+1$ 个不同节点的同意信息后,更新当前共识系统最新节点数量,并将确认信息在系统中广播,此时完成 node-quit 的退出过程。

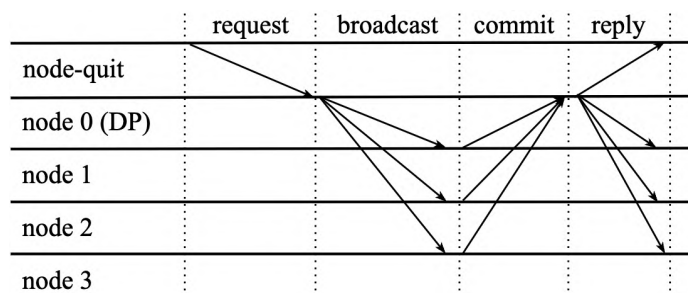


图8 已有节点退出过程

Fig. 8 Quit process of existing nodes

2.6 熔断机制

熔断机制是为了提高共识系统活性,在PBFT算法中,总节点数 $N=3f+1$,其中 f 是拜占庭节点数量。当共识系统中有超过 $1/3$ 以上的节点出现宕机等意外情况,即存活节点数量 $n < 2f+1$ 时,整个共识系统将会被阻塞。因领导者无法收集到来自 $2f+1$ 个不同节点的相同投票信息,而不得不中断共识进程,集群只能持续等待,直到宕机节点恢复。而在例如物联网场景中,大多物联网设备是集中控制的,因此一旦控制中枢出现问题,共识网络中的物联网设备出现大规模宕机是很有可能^[9],所以设计出熔断机制来应对此种情况很有必要。因此当集群存活节点数 $4 < n < 2f+1$ 时,AD-BFT会根据剩余存活节点数量 n ,将原有集群变为由存活节点组成的新的集群,更新group-id,新的集群的总节点数等于存活节点数量,即 $N'=n$, $N'=3f'+1$,此时只需要收集到来自 $2f'+1$ 个不同节点的相同投票,便可以继续推动共识,能够保证系统的可用性。触发熔断机制的场景分为两种:

1)领导者节点均宕机。如图9所示,假设此时只有node 0~node 3运行正常,领导者节点全部宕机。因此在图中 t_1 时刻,剩余存活节点因长时间无法收到DP的KEEP-ALIVE消息,且没有新任领导者接替,则会自主将视图编号加1,进入熔断机制,在集群中广播REQUEST-VOTE消息拉取选票,尝试竞选领导者,消息包含节点自身ID距离最新稳定检查点的距离,以及基础信息摘要(包括节点ID、group-id、最新的稳定检查点哈希值、最新的区块高度、最新视图编号等)。其他节点收到请求投票消息后对消息内容进行验证,验证通过后即可确认对应节点存活,最终每个节点都可以判断出存活节点数量,并会在VOTE阶段将带有节点自身签名的选票投给距离最小的节点node 2。node 2此时已知存活节点数为4,因此在收集到 $2f+1$ 共3个来自不同节点的相同投票后,node 2成为当前视图的DP,发送NEW-VIEW消息给存活节点,更新共识系统总节点数量,此时熔断机制完成,再依次选举出BDP与BBDP后即可恢复共识过程。

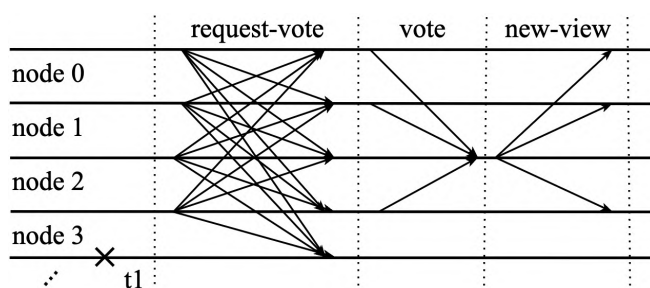


图9 熔断机制场景一

Fig. 9 Circuit breaker mechanism: scenario 1

2)领导者DP存活。如图10所示,假设此时DP存活,超过原有总节点数的 $1/3$ 副本节点出现宕机,在 t_2 时刻DP发现无法继续推动共识,则DP在共识系统中广播PING消息触发熔断机制,探测存活节点数量。副本节点收到消息并在验证确认DP身份后,将节点自身基础信息摘要签名后打包为PONG消息发送至DP,此时DP根据收到的回复信息更新共识系统总节点数量,增加视图号并广播NEW-VIEW消息。若BDP或BBDP出现宕机情况,则根据需要重新选举BDP或BBDP,领导者全部选举完成后进入正常共识过程,此时熔断机制完成。

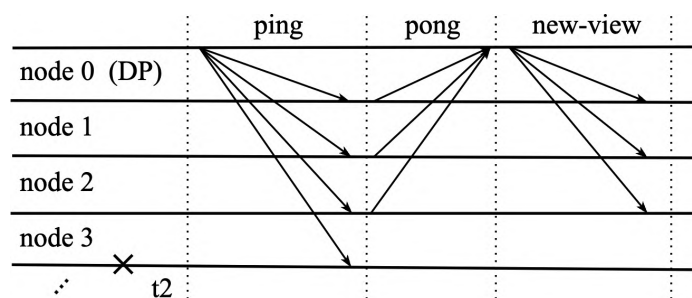


图10 熔断机制场景二

Fig. 10 Circuit breaker mechanism: scenario 2

3 安全性分析

3.1 节点选举

AD-BFT算法运用VRF进行主节点选取,以XOR距离为选举参考值,增强选举安全性与公平性。以下是节点选举安全性分析。

在DP选举过程中,每个副本节点需要计算自身ID哈希值与最新稳定检查点的哈希值之间的XOR距离。哈希函数具有单向性和抗碰撞性。单向性意味着从哈希值难以推理出原始数据,这保证了节点无法通过伪造自身ID哈希值来获取不正当的优势。抗碰撞性确保了不同的输入极大概率产生不同的哈希值,从而保证了计算距离 L 的唯一性和公平性。节点在收到请求投票消息后,根据自身的距离与收到的拉票信息中的距离进行对比来决定投票与否。若自身距离更小,则拒绝投票并记录对方ID,这一规则防止了恶意节点随意投票,保证了投票的合理性。若对方距离更小,则将自己与所收集到的投票信息共同投给这个节点,并且已经投过票的节点不得再给其他节点投票,这避免了重复投票和投票的恶意操纵。只有当某个节点所收集到的投票信息超过共识系统总节点数的 $2/3$ 时,这个节点才能成为DP,这保证了选举出的DP具有广泛的代表性和合法性。在DP选举出来之后,立刻使用VRF生成随机数,VRF算法具有可验证性和随机性的特点。可验证性使得其他节点可以验证随机数的正确性,随机性保证了随机数的不可预测性,并且BDP的身份不会公开,从而防止了恶意节点提前预知。由于BDP本身是通过安全的选举过程产生的,在这个基础上进行的BBDP选举,其选举结果也是相对安全的。在一个视图内,BDP与BBDP的身份不会被公开,当DP宕机时,BDP会立刻广播DP初始生成的随机数,其他节点可以通过验证随机数来确认BDP的合法性,这保证了视图切换过程中的安全性,防止恶意节点冒充BDP进行非法操作。综上,AD-BFT算法中的领导者选举过程在各个环节都有相应的安全保障机制,保证了节点选举的安全性。

3.2 日志一致性

要保证日志的一致性,只需要所有节点在同样的初始条件下,按照相同的顺序执行同样的操作即可。以下是日志一致性分析。

B_m 表示日志块,其中 m 为正整数且 $m > 0$, H_i 表示哈希序列,其中 i 为正整数且 $i > 0$ 。 $h(B_m)$

为哈希函数,表示 B_m 的 SHA256 哈希值。当 $m=i$ 时,可得出。

$$H_i = h\left(h\left(h\left(h(B_1), B_2\right), \dots\right), B_m\right)。(5)$$

若存在恶意节点将其中某个日志块 B_j 篡改改为 B'_j , 其中 $j < m=i$, 得到的 H'_i 为

$$H'_i = h\left(h\left(h\left(h\left(h(B_1), B_2\right), \dots\right), B'_j\right), \dots\right), B_m)。(6)$$

由于哈希函数具有抗碰撞性,因此必然存在 $H_i \neq H'_i$, 所以每个节点都可判断出某份日志被篡改导致 H_i 改变,拒绝提交此日志,保证了日志的一致性。

4 实验结果与分析

本章节对提出的 AD-BFT 与 PBFT 和 DCBFT 进行了仿真模拟,进行对比分析与讨论。实验环境配置为:M2 处理器;MacOS Ventura 13.6.7 操作系统;内存 8 GB;实验部署环境为 Python 3.11.0,使用多线程异步运行在不同的端口来模拟不同的节点,并使用 matplotlib.pyplot 扩展包来绘制实验结果。实验分析对比了通信量,并主要对 PBFT、DCBFT 与 AD-BFT 算法的吞吐量和共识时延进行了测试。

4.1 通信量分析

4.1.1 通信量分析

假设共识系统当前共有 n 个副本节点,在 PBFT 算法的一致性协议中,pre-prepare 阶段的通信量为 $n-1$,prepare 阶段的通信量为 $(n-1)^2$,commit 阶段的通信量为 $n(n-1)$,因此一致协议过程的通信量为 $2n(n-1)$ 。在 PBFT 算法的视图切换协议中,通信量为 $n(n-1)$,假设共识系统发生视图切换的概率为 p_1 ,所以 PBFT 算法执行 r 轮共识的总通信量 W_p 为

$$W_p = nr(2 + p_1)(n-1)。(7)$$

由于 AD-BFT 底层直接使用 PBFT,所以它的一致性协议阶段通信量同 PBFT 一致,为 $2n(n-1)$ 。假设发生视图切换的概率为 p_2 ,则 AD-BFT 中视图切换 1 过程的通信量为 $p_2(n-1)$,视图切换 2 过程的通信量为 $p_2^2(n-3)(n-1)$,所以 AD-BFT 算法执行 r 轮共识的总通信量 W_a 为

$$W_a = r(n-1)(2n + p_2 + p_2^2(n-3))。(8)$$

4.1.2 通信量对比

本节实验中设定共识轮次 $r=1000$,使得 $p_1=p_2=p$,分别取 $p=1$ 和 $p=0.5$ 。实验结果如图 11 所示:随着共识节点数量的增加,PBFT 与 AD-BFT 算法的通信量均明显上升,且 AD-BFT 始终低于 PBFT 的通信量。随着视图切换概率变大,PBFT 与 AD-BFT 之间的通信量差距也明显增加。

4.2 时延测试

共识机制的视图切换时延和共识时延均与共识效率有关,视图切换时延和共识时延越小,则共识效率越高。本节实验通过设定 20 ms 通信延迟与 1% 的丢包率来模拟现实中的网络环境,客户端共发送 1000 笔交易请求,每完成 50 笔交易更新一次稳定检查点,其中 1/3 为拜占庭节

点,发生视图切换的概率 $p=1$,每种节点数量的测试结果取10次实验数据的平均值,以此来保证实验结果的一般性。

视图切换时延是指从出现领导者宕机到完成视图切换所需的时间间隔。由图12可知,AD-BFT的视图切换时延相较于PBFT和DCBFT分别下降了68.4%和35.7%,AD-BFT可以更快的完成视图切换。随着节点数量的不断增加,PBFT的视图切换时延呈明显增长,而AD-BFT的视图切换时延基本稳定,呈小幅上升趋势,始终低于PBFT。AD-BFT视图切换时延最低是因为改进了视图切换过程,BDP在监测到当前视图中的DP宕机后,只需广播NEW-VIEW消息,普通节点收到消息验证BDP的身份后,即可完成视图切换。实验结果表明,在视图切换时延方面,AD-BFT优于PBFT和DCBFT。

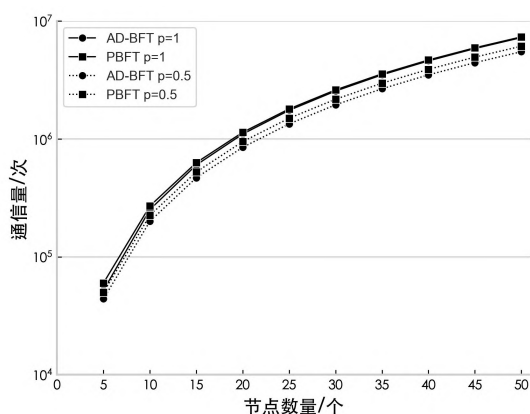


图11 通信量对比

Fig. 11 Communication overhead comparison comparison

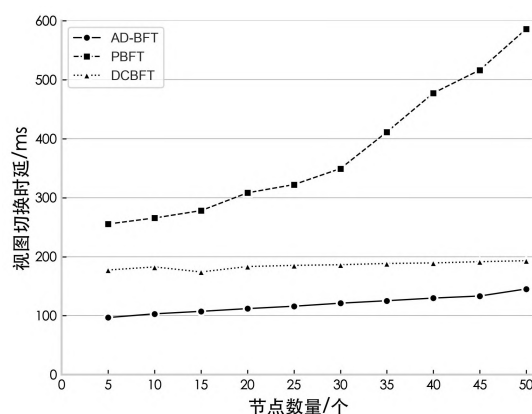


图12 视图切换时延

Fig. 12 View change latency

共识时延是指从客户端发起交易请求后到这笔交易完成上链所消耗的时间^[20]。由图13可知,AD-BFT的共识时延相较于PBFT和DCBFT分别降低了50.9%和9.1%。随着共识节点的数量增加,共识时延均出现上升趋势。由于DCBFT需要所有组内完成共识后,组与组之间还要进行一次共识,因此两阶段的共识比一个阶段的共识消耗时间更多。实验结果表明,在共识时延方面,AD-BFT优于PBFT和DCBFT。

4.3 吞吐量测试

共识机制的吞吐量是指共识算法在一段时间内所能处理的交易数量的大小,是衡量算法性能的一个重要指标,交易吞吐量越高,则共识效率越高。共识通信量与共识时延是影响吞吐量的主要因素。本节实验对比了AD-BFT、PBFT、DCBFT共识的吞吐量,设定节点数为10,其他条件不变,实验结果如图14示。

由实验结果可知,AD-BFT的吞吐量相较于PBFT和CDBFT分别提升了97.2%和9.3%。AD-BFT共识优化了视图切换过程,因此在视图切换频繁的情况下,可以有效降低共识通信量,吞吐量更高,提高共识效率。实验结果表明,在吞吐量方面,AD-BFT优于PBFT和DCBFT。

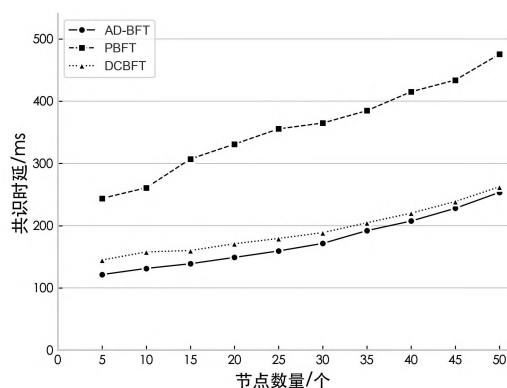


图13 共识时延

Fig. 13 Consensus latency

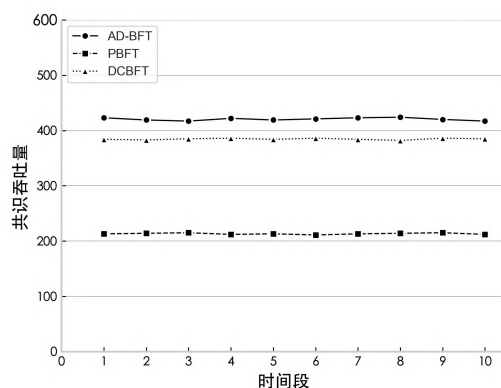


图14 吞吐量

Fig. 14 Throughput

4.4 动态进出测试

动态进出过程所需要的时间是衡量AD-BFT的一项指标,本节实验分别测试了从30个节点依次增加至50个节点和从50个节点依次减少到30节点这两个过程的进出时延变化。每添加或减少2个节点的平均时延结果如图15所示。

实验结果显示,随着共识节点数量的增加,节点加入共识系统所需时间明显上升,这是因为节点数量增多,加入过程所需要发送的广播消息增多。在依次减少共识节点数量时,节点数量减少,所需发送的广播消息变少,因此退出时延降低。整体来看,加入或退出时间均小于50 ms,远低于同等数量下的共识时延,对共识过程影响较小,并且由于加入过程需要数据同步,所以加入过程时延略高于同等数量下的退出时延。实验结果显示,AD-BFT可以有效处理节点动态进出事务。

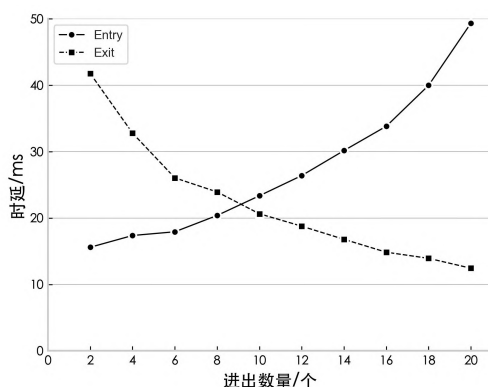


图15 动态进出时延

Fig. 15 Dynamic entry and exit Latency

4.5 熔断机制测试

熔断机制触发后的恢复时延是衡量算法的另一项重要指标,在AD-BFT中表示集群大量节点宕机到恢复至正常共识的时间差。本节实验使用25个节点,在共识完成500笔交易后分别宕机40%、60%、80%的节点数量,其他环境因素均不变,计算第500笔交易完成到三个领导者节点全部重新选举完成的时间差,最后随机选取测试结果中的10组数据进行比较,得出熔断恢复测试结果如图16所示,每共识完成100笔交易的平均吞吐量结果如图17所示。

根据实验结果可知,AD-BFT可在400 ms以下的时间内恢复共识,这一时间相当于PBFT完成一次共识的时间。随着节点宕机比例增加,收集选票所需要的时间减少,因此恢复时间也明显下降。当共识系统出现大量节点宕机后,由于共识节点数量减少,因此在恢复后,吞吐量出现明显增加。所以熔断机制保证了AD-BFT在节点出现大量宕机时能够继续共识,提高了系统的健壮性。

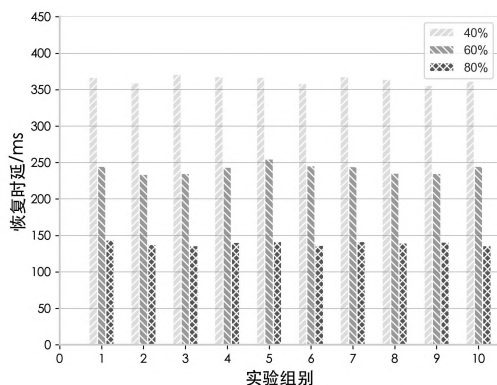


图16 熔断恢复时延

Fig. 16 Circuit breaker recovery latency

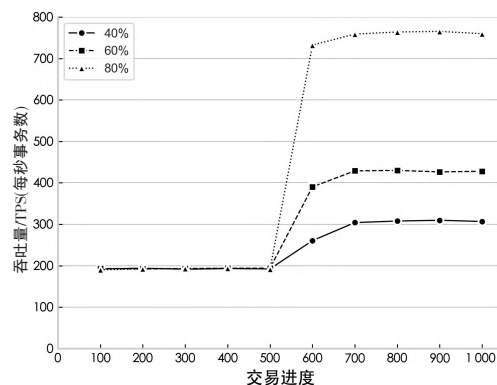


图17 熔断吞吐量

Fig. 17 Circuit breaker throughput Throughput

5 结论

基于PBFT提出了AD-BFT共识算法。首先,基于VRF优化领导者节点的选举过程,随机选择新一任期领导者,可以避免预谋攻击,增强系统安全性。改进视图切换流程,减少了视图切换的通信量与共识时延,提高系统吞吐量。其次,设计并实现了节点动态进出机制与熔断机制来提高系统可扩展性与健壮性。最后,试验结果与分析表明,AD-BFT相较于PBFT,有着更低的共识时延与更大的吞吐量。

参考文献:

- [1] NAKAMOTO S. Bitcoin: A peer-to-peer electronic cash system[EB/OL]. (2018-04-10)[2023-03-09]. <https://bitcoin.org/bitcoin.pdf>.
- [2] 周炜,王超,徐剑,等. 基于区块链的隐私保护去中心化联邦学习模型[J]. 计算机研究与发展, 2022, 59(11): 2423-2436.
- [3] 刘教迪,杜学绘,王娜,等. 区块链系统安全防护技术研究进展[J]. 计算机学报, 2024, 47(3): 608-646.
- [4] 邵奇峰,金澈清,张召,等. 区块链技术: 架构及进展[J]. 计算机学报, 2018, 41(5): 969-988.
- [5] 沈鑫,裴庆祺,刘雪峰. 区块链技术综述[J]. 网络与信息安全学报, 2016, 11(2): 11-20.
- [6] NAVARRO J G I, JULIE E G, ROBINSON Y H. Adaptive practical byzantine fault tolerance consensus algorithm in permission blockchain network[J]. International Journal of Web and Grid Services, 2022, 18(1): 62-82.
- [7] DING Z, DONG X, SHEN J, et al. A hybrid double-layer BFT consensus protocol for large-scale IoT blockchain [C]// 2nd International Conference on Frontiers of Electronics, Information and Computation Technologies(ICFE-ICT). IEEE, 2022: 354-361.
- [8] LIU X, LIU Y, LI X, et al. FP-BFT: A fast pipeline byzantine consensus algorithm[J]. IET Blockchain, 2023, 3

- (3): 123-135.
- [9] TANG Y, YAN J, CHAKRABORTY C, et al. Hedera: A permissionless and scalable hybrid blockchain consensus algorithm in multiaccess edge computing for IoT[J]. IEEE Internet of Things Journal, 2023, 10(24): 21187-21202.
- [10] GUO R, GUO Z, LIN Z, et al. A hierarchical byzantine fault tolerance consensus protocol for the internet of things[J]. High-Confidence Computing, 2024, 4(3): 100196.
- [11] HAO R, DAI X, XIE X. Doppel: A bft consensus algorithm for cyber-physical systems with low latency[J]. Journal of Systems Architecture, 2024, 148: 103087.
- [12] XIAO J, LUO T, LI C, et al. CE-PBFT: A high availability consensus algorithm for large-scale consortium blockchain[J]. Journal of King Saud University-Computer and Information Sciences, 2024, 36(2): 101957.
- [13] FATHI F, BAGHANI M, BAYAT M. Light-PerIChain: Using lightweight scalable blockchain based on node performance and improved consensus algorithm in IoT systems[J]. Computer Communications, 2024, 213: 246-259.
- [14] 唐宏, 刘双, 酒英豪, 等. 实用拜占庭容错算法的改进研究[J]. 计算机工程与应用, 2022, 58(9): 144-150.
- [15] 雷元娜, 徐海霞, 李佩丽, 等. 区块链共识机制中随机性研究[J]. 信息安全学报, 2021, 6(3): 91-105.
- [16] MAYMOUNKOV P, MAZIERES D. Kademlia: A peer-to-peer information system based on the XOR metric[J]. Lecture Notes in Computer Science, 2002, 2429(1): 53-65.
- [17] SCHIPER A. Dynamic group communication[J]. Distributed Computing, 2006, 18(5): 359-374.
- [18] DUAN S, ZHANG H. Foundations of dynamic bft[C]//IEEE Symposium on Security and Privacy(SP). IEEE, 2022: 1317-1334.
- [19] SI X, LI M, YAO Z, et al. An efficient and secure blockchain consensus protocol for internet of vehicles[J]. Electronics, 2023, 12(20): 4285.
- [20] 杨保华, 陈昌. 区块链原理、设计与应用[M]. 北京: 机械工业出版社, 2017.

[责任编辑: 钟国翔]